

IT: cloud-server workload

Python model: `plant_models.cloud_server_workload`

Description

Aggregate cloud-server backlog with saturating service and actuator lag.

Controlled input and output

Role	Mathematical notation	Python/interface name	Meaning	Unit
Input	u	<code>u</code>	Compute-capacity command	•
Output	$\Delta\tau$	<code>y / latency_deviation</code>	Response-time deviation	s

State equations

The physical state vector used by the model is

$$\chi = [x, c]^\top.$$

The auxiliary quantities are

$$\begin{aligned}c_{\text{cmd}} &= c_0 + k_c \tanh(u) \\ s(x, c) &= \frac{[c]_+ [x]_+}{K_s + [x]_+} \\ [r]_+ &= \max(r, 0)\end{aligned}$$

The implemented continuous-time dynamics are

$$\begin{aligned}\dot{x} &= \lambda - s(x, c) - k_a x \\ \dot{c} &= \frac{c_{\text{cmd}} - c}{\tau_c}\end{aligned}$$

The implementation clips $\dot{x}$ at zero when $x \leq 0$ and the unconstrained derivative is negative.

Parameters

The mathematical notation is used in the equations above. The Python name is the exact field in `PlantParams`; the last column explains how the parameter enters this specific model.

Mathematical notation	Python name	Default	Meaning
x_0	<code>backlog_nom</code>	8.0	Initial or nominal pending-work backlog.
λ	<code>arrival_rate</code>	5.0	External work-arrival rate.
c_0	<code>service_nom</code>	5.0	Nominal server service rate.
k_c	<code>service_gain</code>	3.0	Gain from normalized control input to additional service capacity.
τ_c	<code>tau_capacity</code>	1.5	First-order time constant of provisioned server capacity.

Mathematical notation	Python name	Default	Meaning
K_s	half_saturation	2.0	Backlog at which the nonlinear service-utilization term reaches half of its asymptotic value.
k_a	abandonment	0.03	Rate at which queued work abandons or expires.
k_L	latency_gain	0.18	Conversion from backlog to reported service latency.

Notation and variables

The first column gives the readable mathematical notation, the second gives the exact Python or SISO-interface name, and the third states the model-specific physical meaning. Thus the equations remain readable while every symbol can be traced back to the implementation.

Mathematical notation	Python name	Meaning
u	u	Compute-capacity command [-]
$\Delta\tau$	y / latency_deviation	Response-time deviation [s]
x	backlog	Pending workload [jobs]
c	allocated_capacity	Allocated compute capacity [jobs/s]
μ	service_rate	Completed workload rate [jobs/s]
τ	response_time	Response-time proxy [s]
x	x	Position or displacement state specified by this mechanical model.
c	c	Carbon or concentration state defined by the equations.

Implementation reference

Initial state:

```
def initial_state(par): return np.array([par.backlog_nom, par.service_nom, 0.0], flt
```

Algebraic outputs:

```
def algebraic_outputs(chi, par):
    x, c = chi[:2]
    service = c*x/(par.half_saturation+max(x,0.0))
    latency = par.latency_gain*x
    return {"latency_deviation": latency-par.latency_gain*par.backlog_nom,
            "backlog": x, "allocated_capacity": c, "service_rate": service,
            "response_time": latency}
```

ODE right-hand side:

```
def rhs(t, chi, u, par):
    x, c = chi[:2]
    c_cmd = par.service_nom + par.service_gain*np.tanh(float(u))
    service = max(c,0.0)*max(x,0.0)/(par.half_saturation+max(x,0.0))
    dx = par.arrival_rate-service-par.abandonment*x
    if x <= 0.0 and dx < 0.0: dx = 0.0
    return np.array([dx, (c_cmd-c)/par.tau_capacity, 0.0], float)
```